

CS10 Quiz 1

<i>Last Name</i>	
<i>First Name</i>	
<i>Student ID Number</i>	
<i>Name of the person to your Left</i>	
<i>Name of the person to your Right</i>	
<i>All my work is my own. I had no prior knowledge of the exam contents nor will I share the contents with others in CS10 who have not taken it yet. (please sign)</i>	

Instructions

- This booklet contains 4 double-sided pages including this cover page. Put all answers on these pages; don't hand in any stray pieces of paper.
- Please put all smart watches, cell phones, and beepers at the back of the room. Remove all hats, headphones, and watches.
- You have 180 minutes to complete this exam. This final is a closed book, no computers, watches, no PDAs, no cell phones, no calculators. There may be partial credit for incomplete answers; write as much of the solution as you can. When we provide a blank, please fit your answer within the space provided.

Points

Abstraction	Binary - Hex - Dec	Iteration	Domain & Range	Booleans	Functions	HOFs
2	4	6	6	6	4	12

Question 1 - Abstraction (2 Points)

How does moving from **BEFORE** to **AFTER** affect *Abstraction* and its related concepts?

PART A

BEFORE	AFTER
draw square of side 100	draw -sided polygon with edges length

Detail Removal: ☐ decreases ☐ increases ☐ no change

Generalization: ☐ decreases ☐ increases ☐ no change

Abstraction: ☐ decreases ☐ increases ☐ no change

PART B

Specification for Double	
BEFORE	AFTER
Inputs: a number	Inputs: a number
Output: Double the input	Output: Double the input
Example: Double 5 10	Example: Double 5 10
	How we implemented it: We called the input parameter <code>n</code> . We took <code>n</code> , then added it to itself and set a temporary variable <code>answer</code> to that sum. Then we returned that <code>answer</code> variable (which was effectively <code>n + n</code>).

Detail Removal: ☐ decreases ☐ increases ☐ no change

Generalization: ☐ decreases ☐ increases ☐ no change

Abstraction: ☐ decreases ☐ increases ☐ no change

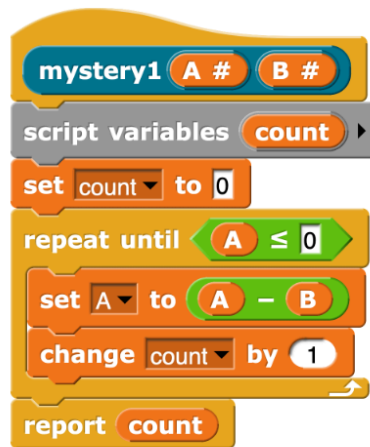
Question 2 - Number Representation (4 Points)

What is $1010_2 + 13_{10}$?

- ☐ 25_{16}
- ☐ $1F_{16}$
- ☐ 17_{16}
- ☐ $1D_{16}$
- ☐ 21_{16}
- ☐ $1B_{16}$
- ☐ 19_{16}
- ☐ 23_{16}

Question 3 - Iteration (6 points)

Part A



- a) Given the code above, what does **mystery1** **A** **B** report when **A** is set to 12, and **B** is set to 3? (Fill in the blank)

Answer: _____

Part B

b) What does `mystery2` compute if `A` and `B` are positive integers?



- ☐ `A - B`
- ☐ `B - A`
- ☐ The minimum value of `A` and `B`
- ☐ The maximum value of `A` and `B`
- ☐ The number of times `B` can be added up before it is bigger than `A`
- ☐ The number of times `A` can be added up before it is bigger than `B`
- ☐ The remainder when `A` is divided by `B`
- ☐ The remainder when `B` is divided by `A`
- ☐ None of the Above

Question 4 - Domain and Range (6 points)

If `outer Function inner value` is error-free, with the following properties:

- `inner range`: word
- `outer range`: list
- `inner domain`: Boolean
- `outer domain`: character

...and we want `new Function input` to be error-free, which of the following types would be allowed? (Don't forget about subtypes! For example, if *numbers* were allowed, then a *digit* would **also** be allowed, since a *digit* is also a *number*.) `new range`:

☐ Boolean ☐ list ☐ sentence ☐ word ☐ character ☐ letter

`new domain`: ☐ Boolean ☐ list ☐ sentence ☐ word ☐ character ☐ letter

`input`: ☐ Boolean ☐ list ☐ sentence ☐ word ☐ character ☐ letter

Question 5 - Conditional Operators and Booleans (6 points)



Part 1

Suppose the output of **Pred** is **true**.

What can you say for sure about **A**?

- ☐ **A** must be true
- ☐ **A** must be false
- ☐ Nothing

What can you say for sure about **B**?

- ☐ **B** must be true
- ☐ **B** must be false
- ☐ Nothing

Part 2

Suppose the output of **Pred** is **false**.

What can you say for sure about **A**?

- ☐ **A** must be true
- ☐ **A** must be false
- ☐ Nothing

What can you say for sure about **B**?

- ☐ **B** must be true
- ☐ **B** must be false
- ☐ Nothing

Part 3

Which definition below computes the same logical function as **Pred** at the top?

☐

```
Pred A B
if A
  if not B
    report true
  report false
```

☐

```
Pred A B
if not A
  if B
    report true
  report false
```

☐

```
Pred A B
if not A
  if not B
    report true
  report false
```

☐

```
Pred A B
if A
  report true
if not B
  report true
report false
```

☐

```
Pred A B
if not A
  report true
if B
  report true
report false
```

☐

```
Pred A B
if not A
  report true
if not B
  report true
report false
```

Question 6 - Functions (4 points)

If we were given three functions:

$$R(x) = x + 4$$

$$S(x) = x - 3$$

$$K(x) = x \times 2$$

... and you want to calculate:

$$(x - 3 + 4) \times 2$$

... how would you compose the three functions to get that?

- ☐ $R(K(S(x)))$
- ☐ $S(R(K(x)))$
- ☐ $R(S(K(x)))$
- ☐ $K(S(R(x)))$
- ☐ $S(K(R(x)))$
- ☐ $K(R(S(x)))$

Question 7 - HOFs (12 points)

We want to take the command blocks `delete`, `replace`, and `add` and convert them from commands to reporters so that they return *new* lists and don't modify their input list. What would be needed to reimplement them as reporters as shown in each example? (we also add a bonus reporter, called `only the ith elements`).

a)



- ☐ Can be done with 1 **map**
- ☐ Can be done with 1 **keep**
- ☐ Must be done with a **map** and **keep** together
- ☐ None of these

b)



- ☐ Can be done with 1 **map**
- ☐ Can be done with 1 **keep**
- ☐ Must be done with a **map** and **keep** together
- ☐ None of these

c)



- ☐ Can be done with 1 **map**
- ☐ Can be done with 1 **keep**
- ☐ Must be done with a **map** and **keep** together
- ☐ None of these

d)



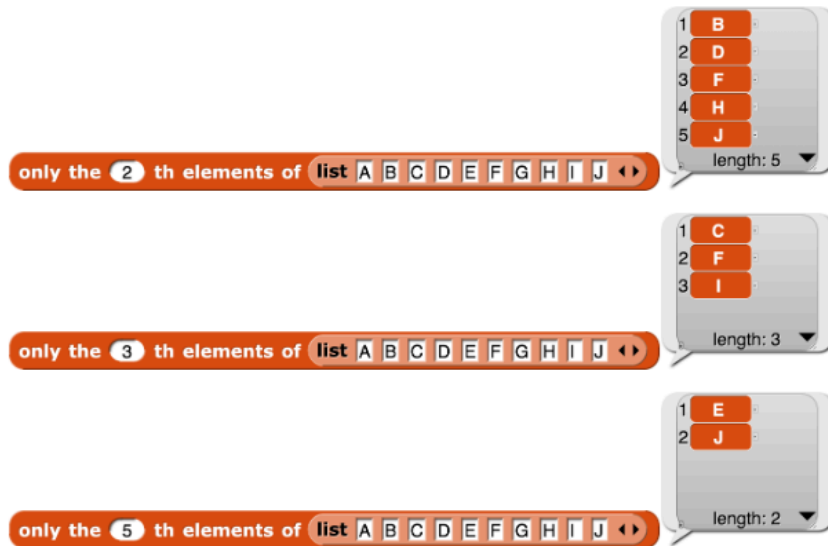
- ☐ Can be done with 1 **map**
- ☐ Can be done with 1 **keep**
- ☐ Must be done with a **map** and **keep** together
- ☐ None of these

e)



- ☐ Can be done with 1 **map**
- ☐ Can be done with 1 **keep**
- ☐ Must be done with a **map** and **keep** together
- ☐ None of these

f)



- ☐ Can be done with 1 **map**
- ☐ Can be done with 1 **keep**
- ☐ Must be done with a **map** and **keep** together
- ☐ None of these